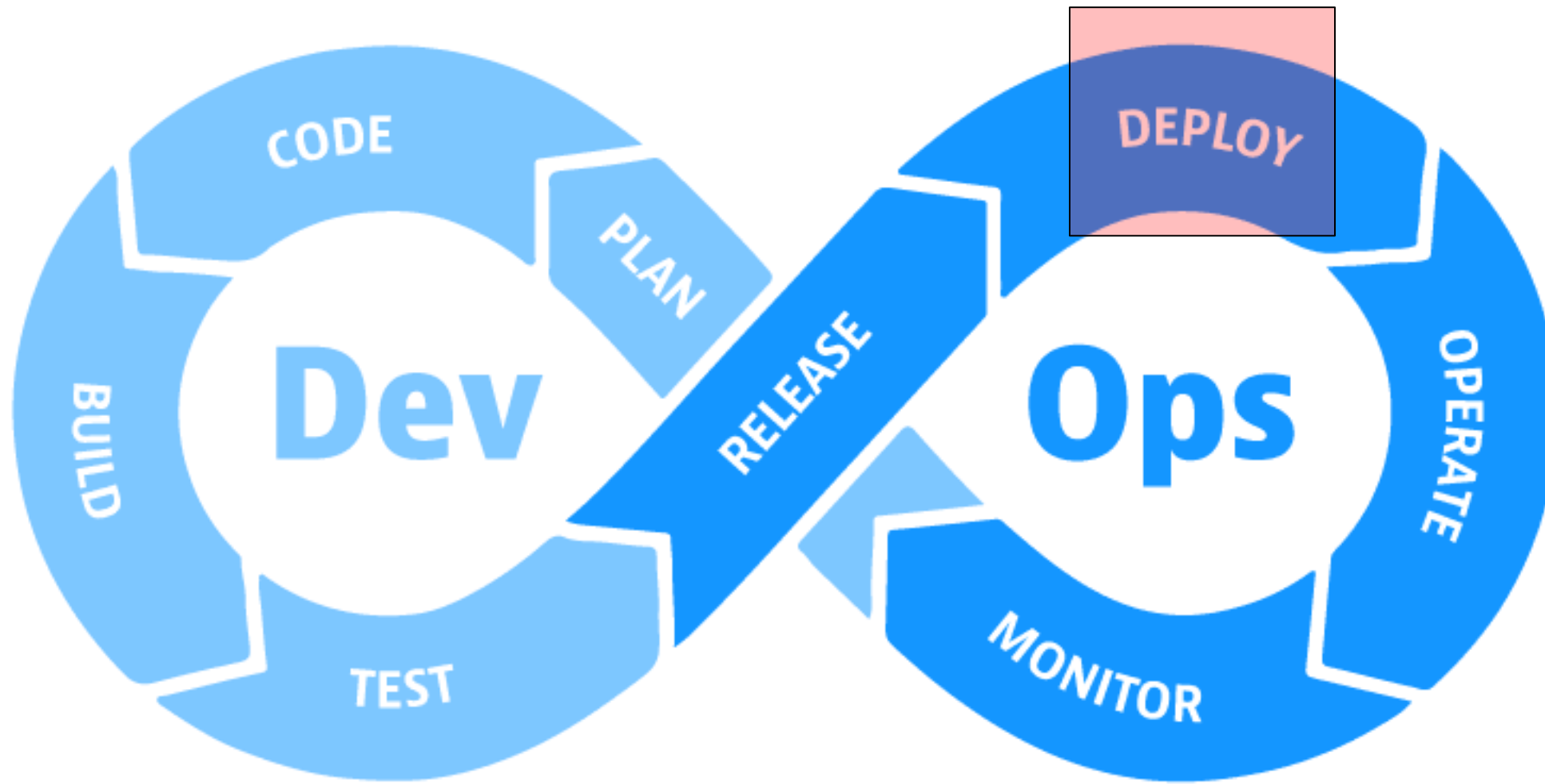


Developer Operations (devops)

Deployment – Helm + Stages



Deployment



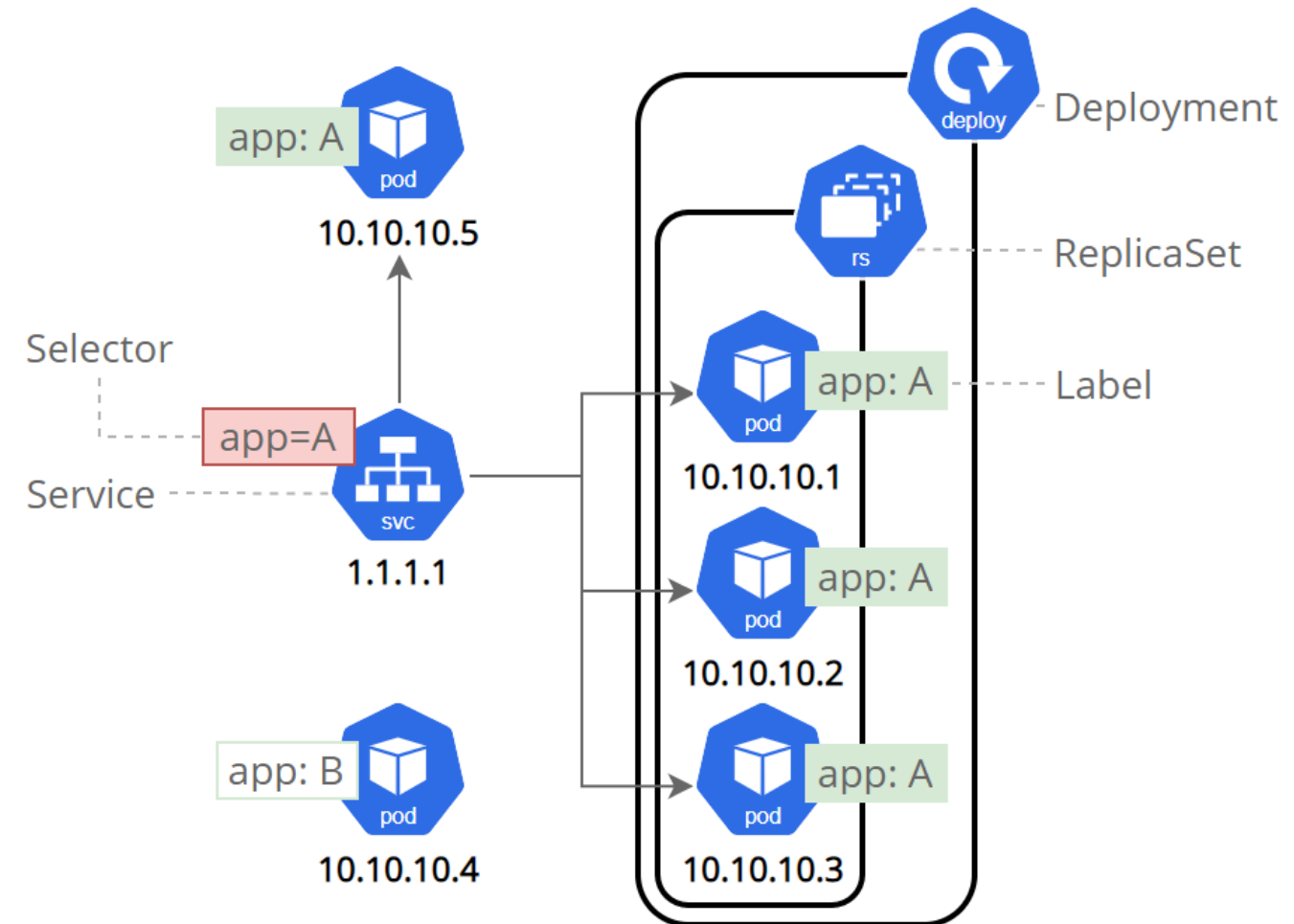
What's it all about today?

- Kubernetes Recap
- Challenges with Kubernetes manifest management
- Kubernetes packaging (e.g. Helm)
- Deployment Stages

Recap, Elements within Kubernetes

Objects, necessary for deploying an app

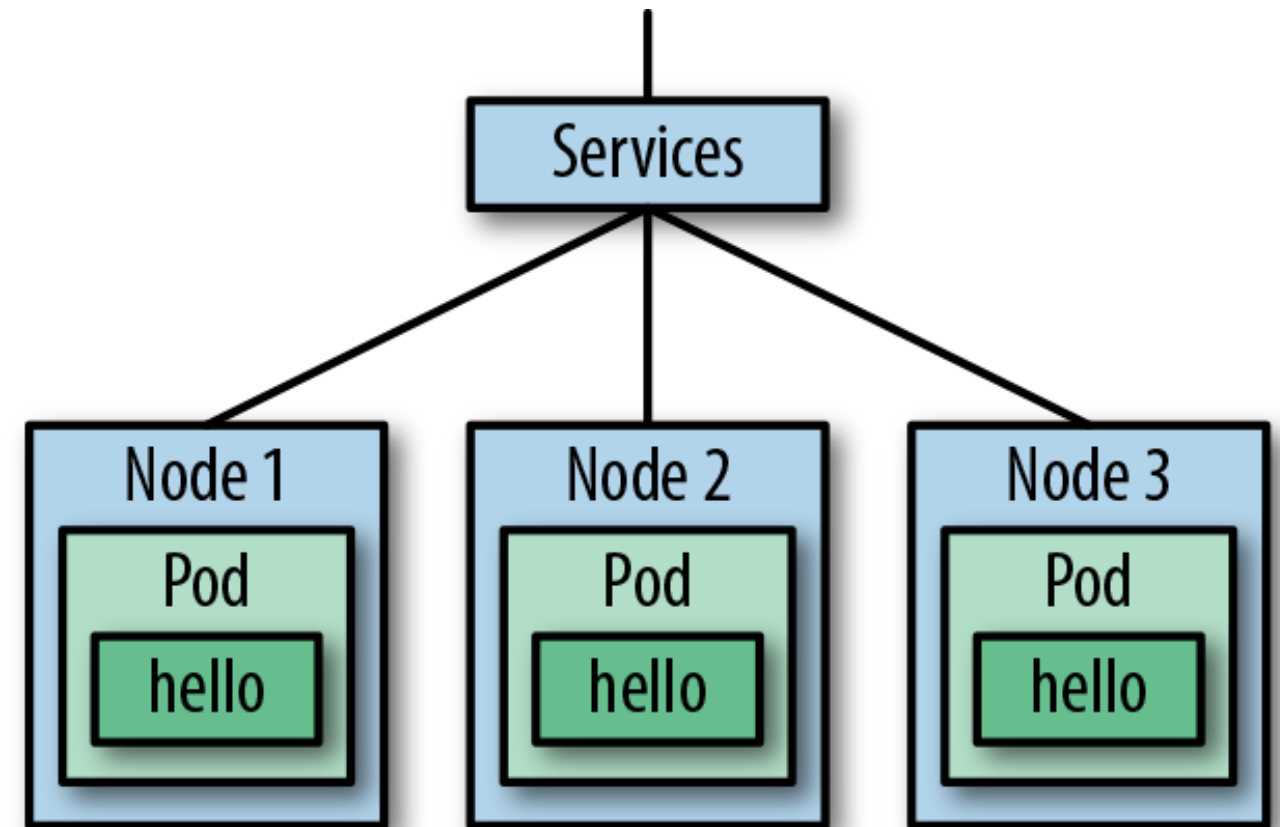
- **Deployment:** Description of target to deploy service
- **ReplicaSet:** Ensuring that pods are running
- **Pods:** Single set of running container
- **Service:** Address + Portmapping to a set to pods defined by label



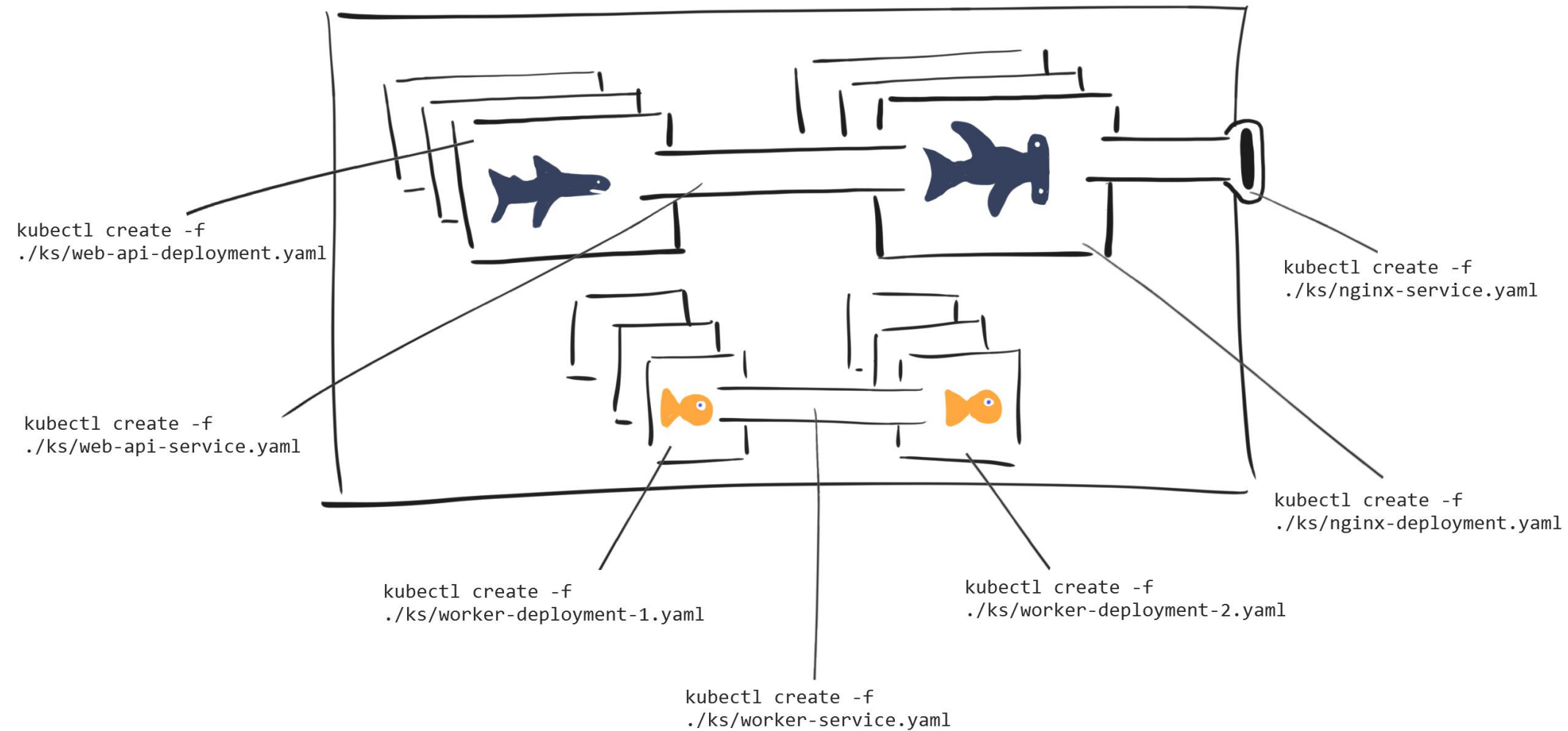
Service

- Traffic is bound to ports and containers
- Containers are ephemeral and might be deleted/created/etc.
- Multiple pods might represent the same service

Abstract is needed: a Service defines an open port forwarding traffic to all pods with a defined label



Kubectl, Problems?



Common Antipatterns within Kubernetes Deployment

1. Putting configuration near the image generation
2. Not using Helm / Kustomize
3. Deploying things in specific order
4. Pulling latest images
5. Deploying due to killing pods

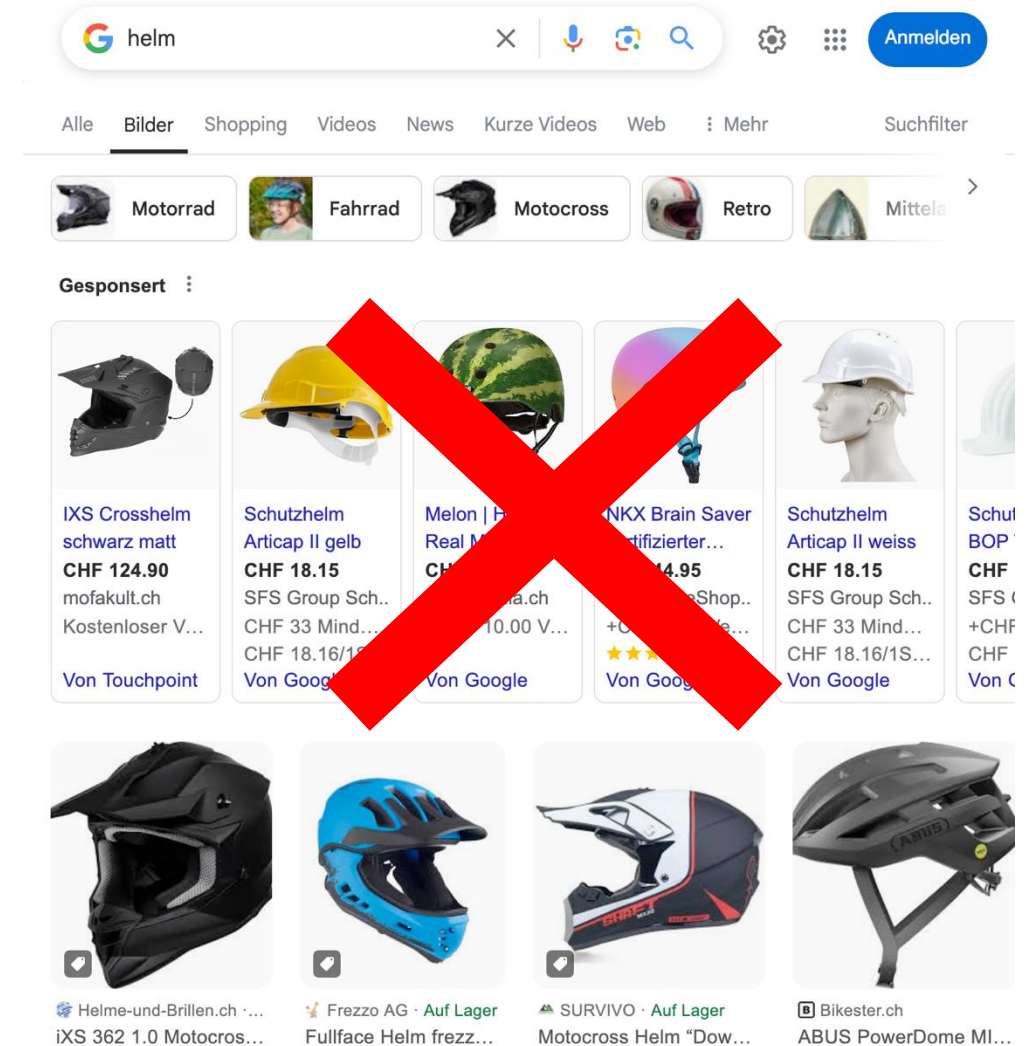


Project Intro

Origin

- Naming
 - Kubernetes: greek for "helmsman" or "pilot"
 - Helm: greek again; „steering mechanism of ship“
- Started 2015 by Deis (now Microsoft Azure)
- Graduated CNCF project (2020)
- De-facto standard for packaging of 3rd party Kubernetes apps

<https://helm.sh/>



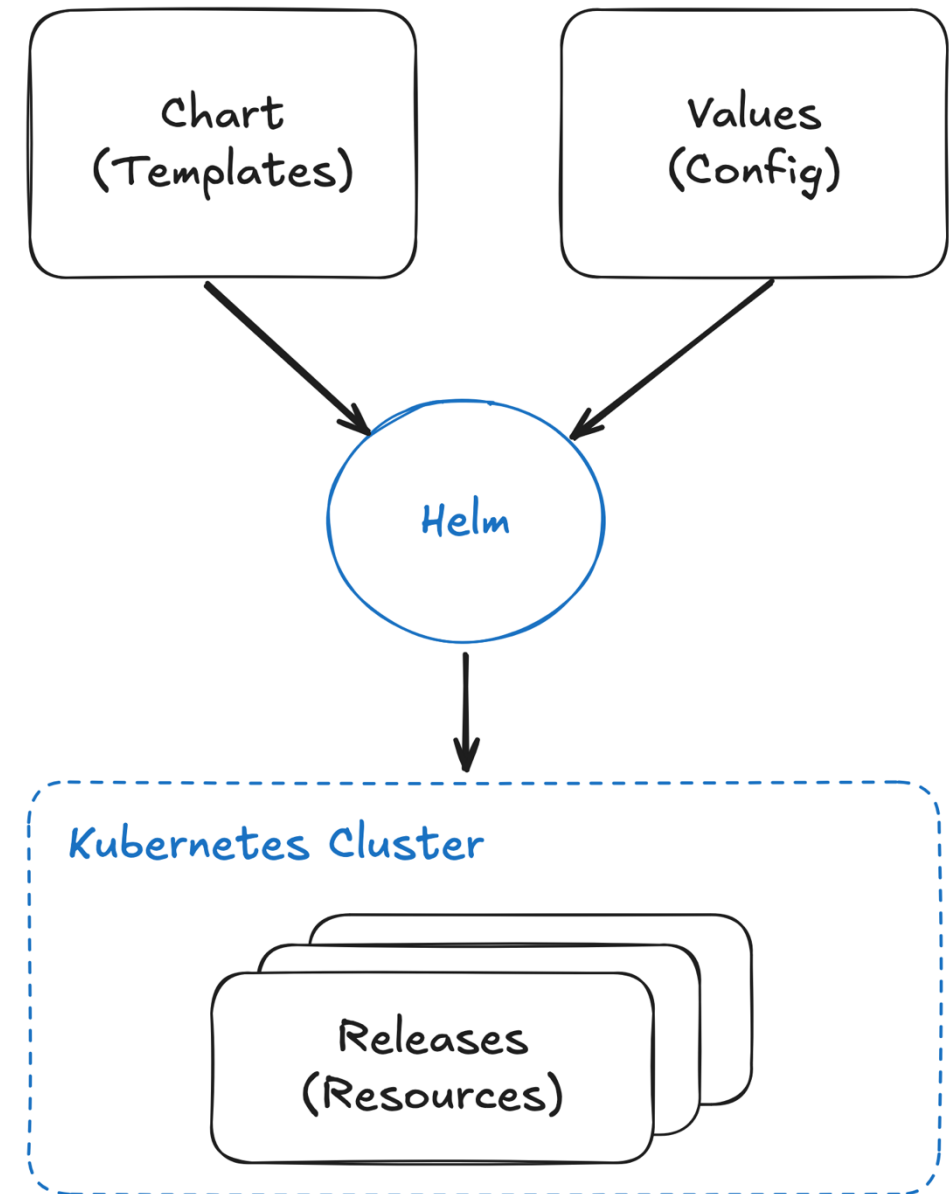
Helm - Concepts

Chart: Helm package, containing all resources to run an application inside Kubernetes

Values: Configuration for one released chart.

Release: Instance of installed chart with provided configuration. Can occur multiple times inside a cluster in different namespaces. Each release has one defined release name.

„Kubernetes resource templates are merged with Values to create resulting Release“

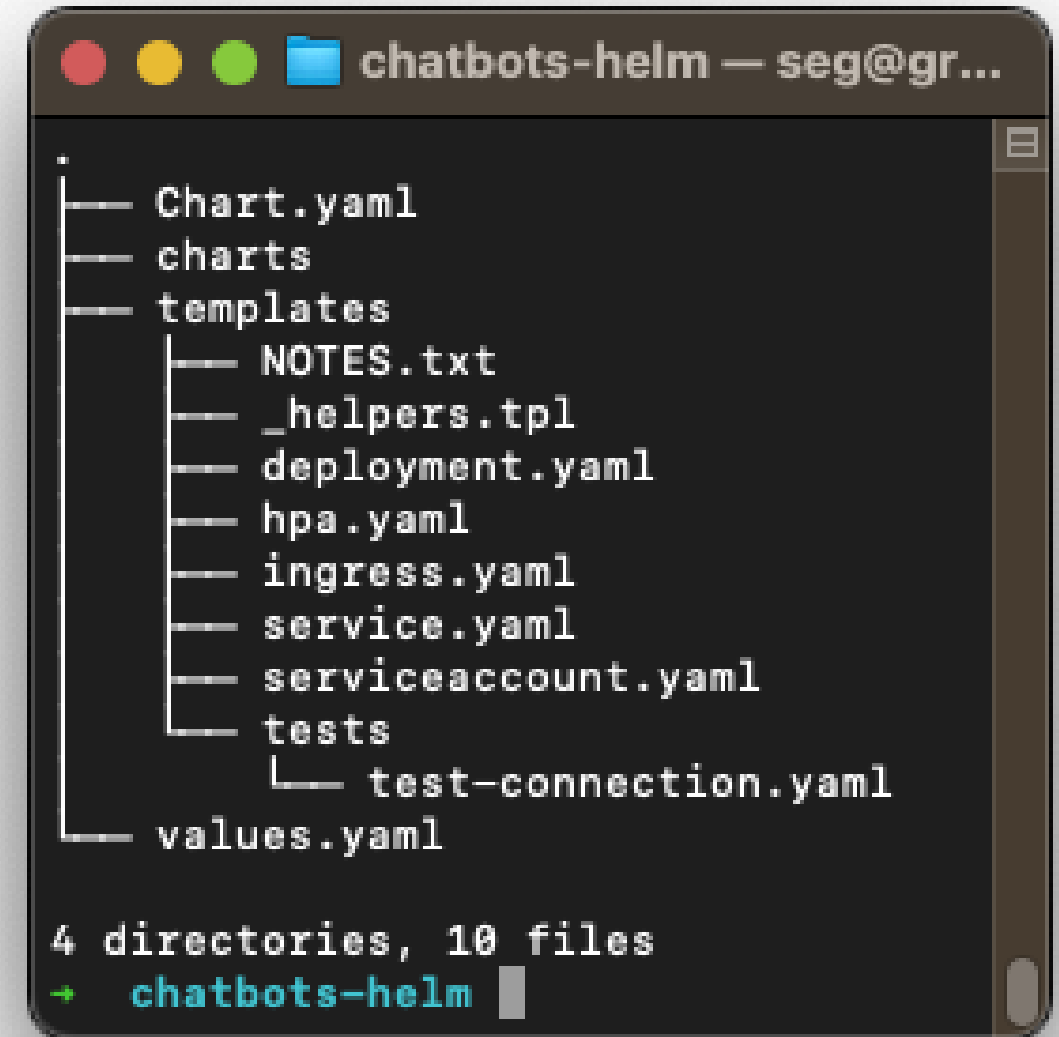


Source: Krusche & Company GmbH

Structure and Usage

Defined folder structure

- templates: K8s-resources to be deployed
 - Should contain entire architecture
 - Different values should be replaced by references
- values.yaml:
 - Replacing values in templates/resources
- Chart.yaml:
 - Metainformation about chart
 - Reference to subcharts
- CLI-Tool
 - Takes folder structure or packaged chart as input



Packaging



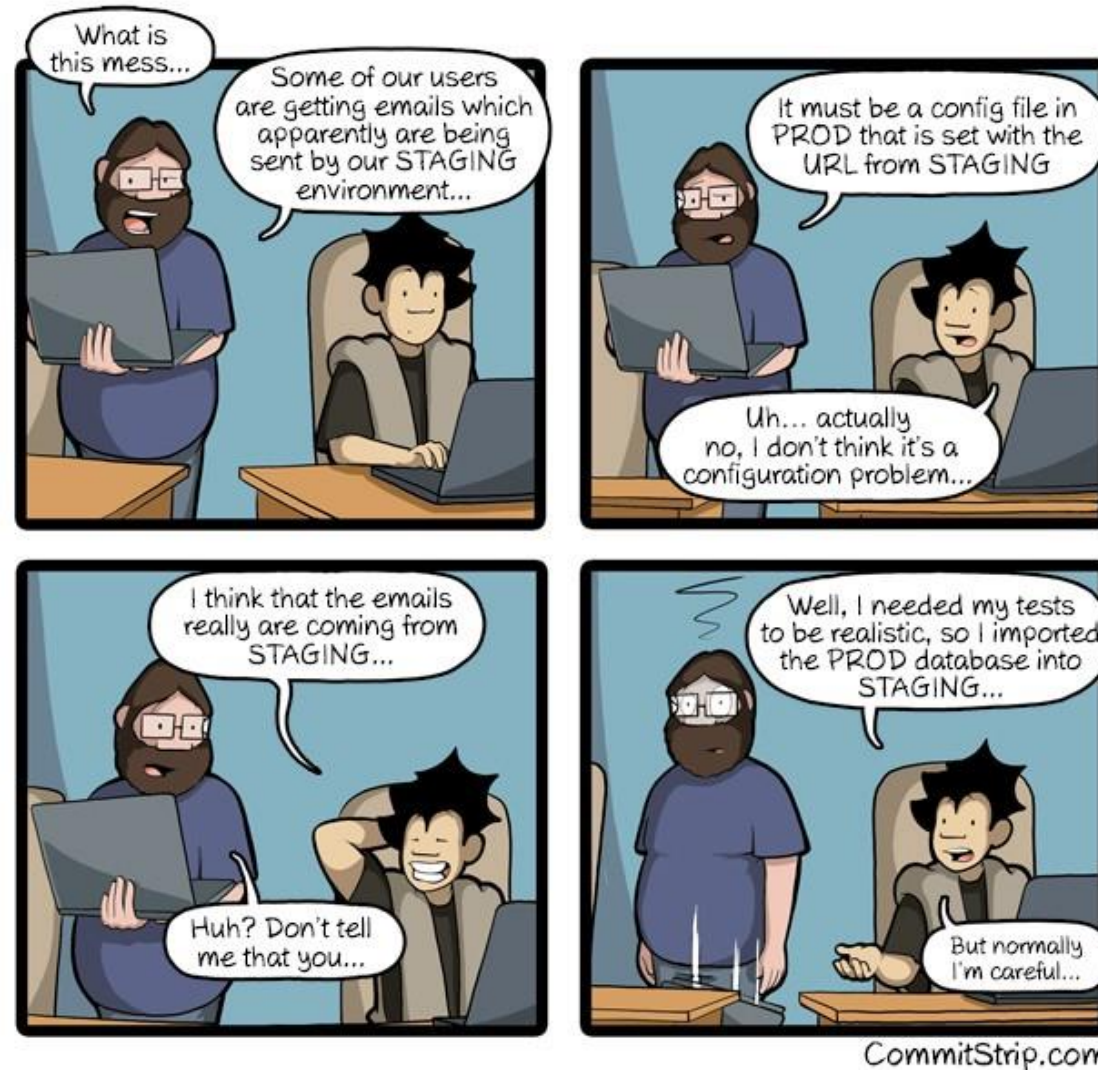
- Helm Charts can be packaged and distributed
 - Well-known public marketplace: <https://artifacthub.io/>
 - Running your own repository, e.g. via <https://chartmuseum.com/>
- Currently two packaging standards exist
 - Helm Repository (original and custom registry format)
 - OCI Registry (stored like container images, available since v3.8.0)
 - Less functionality with local Helm client (missing search, publish etc.)
- Usage
 - `$ helm repo add [NAME] [URL]` # adding 3rd-party repo
 - `$ helm package [CHART_PATH]` # packaging your own chart

Demo



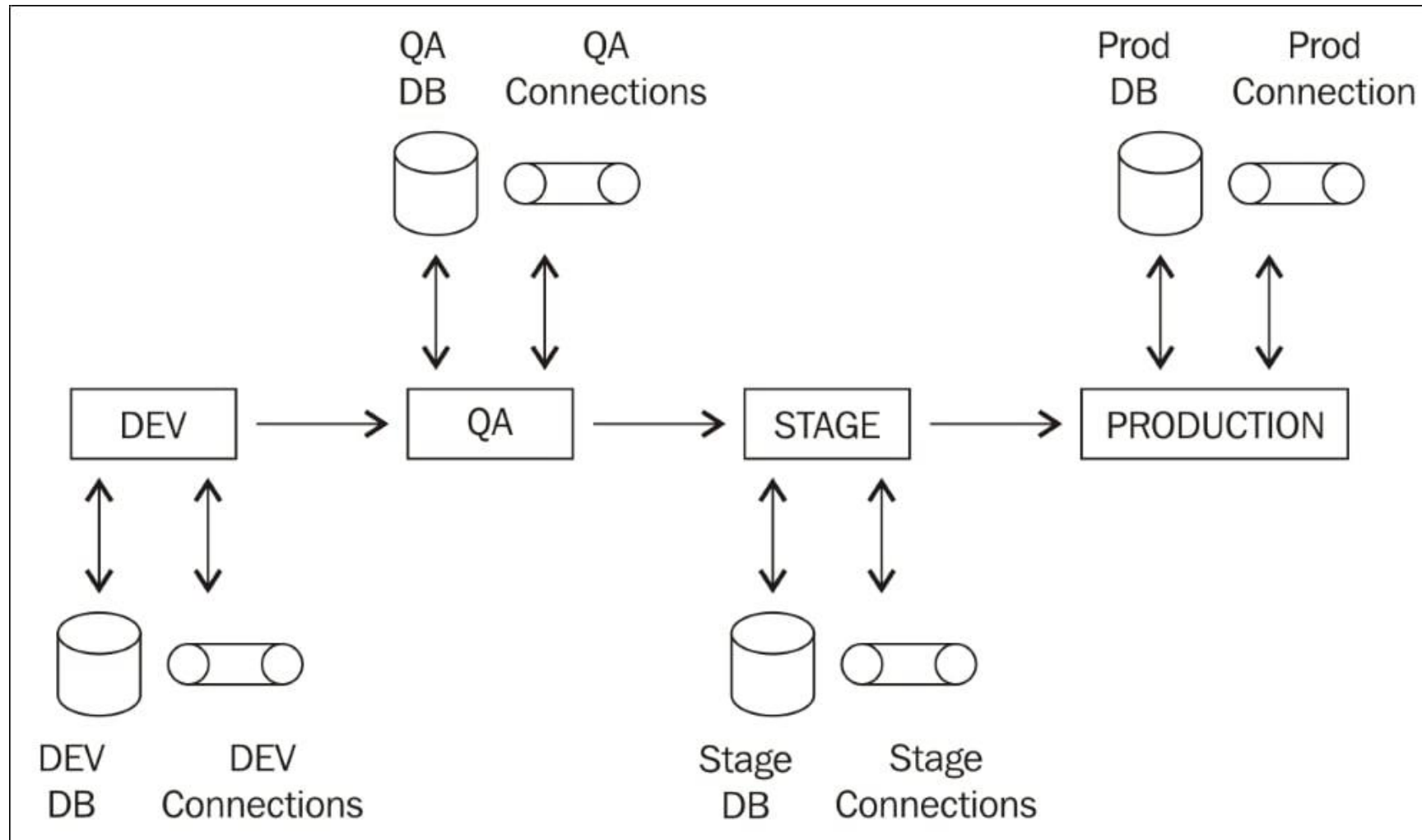
- **Start a new chart**
- **Installing a release**
- Very simple Helm chart can be found here:
 - <https://github.com/cloudnativdevops/demo/tree/main/hello-helm3>

Stages in the Real World

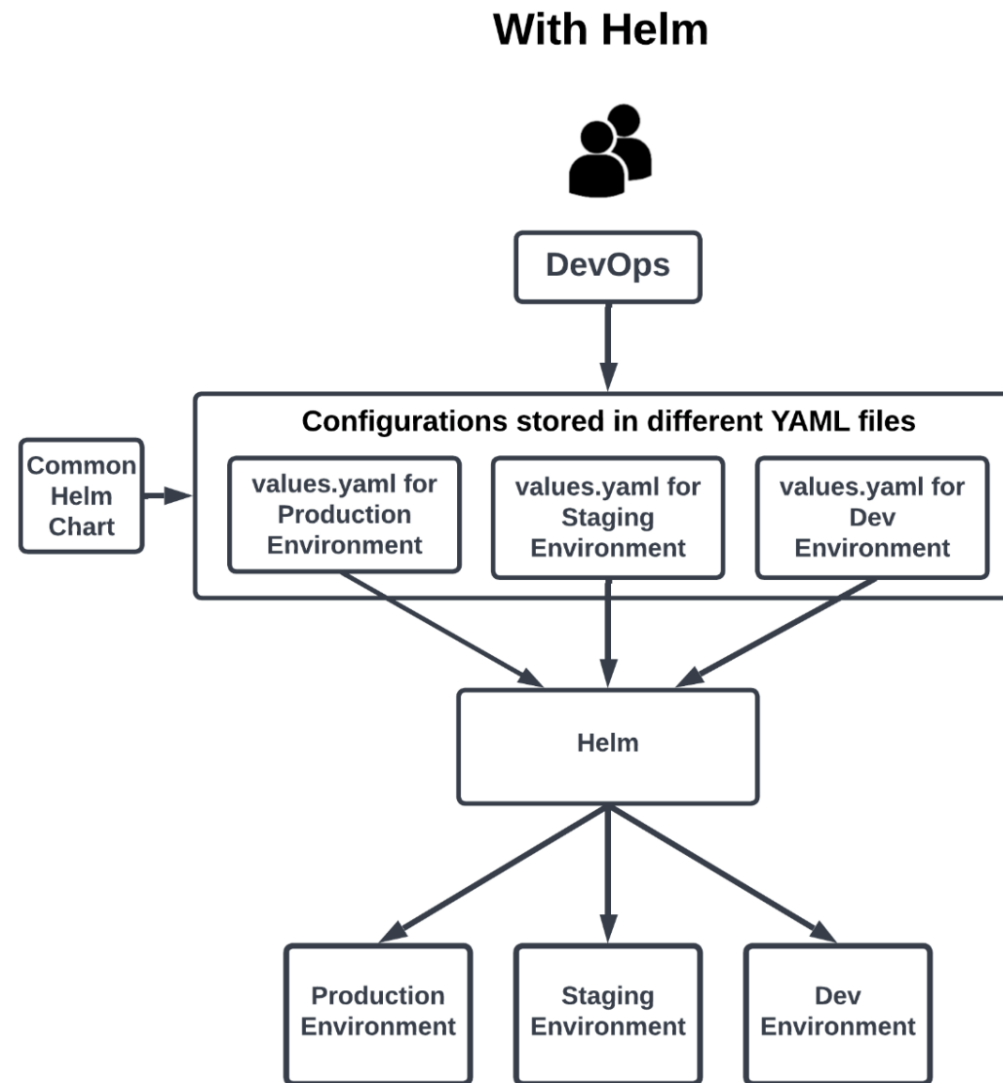
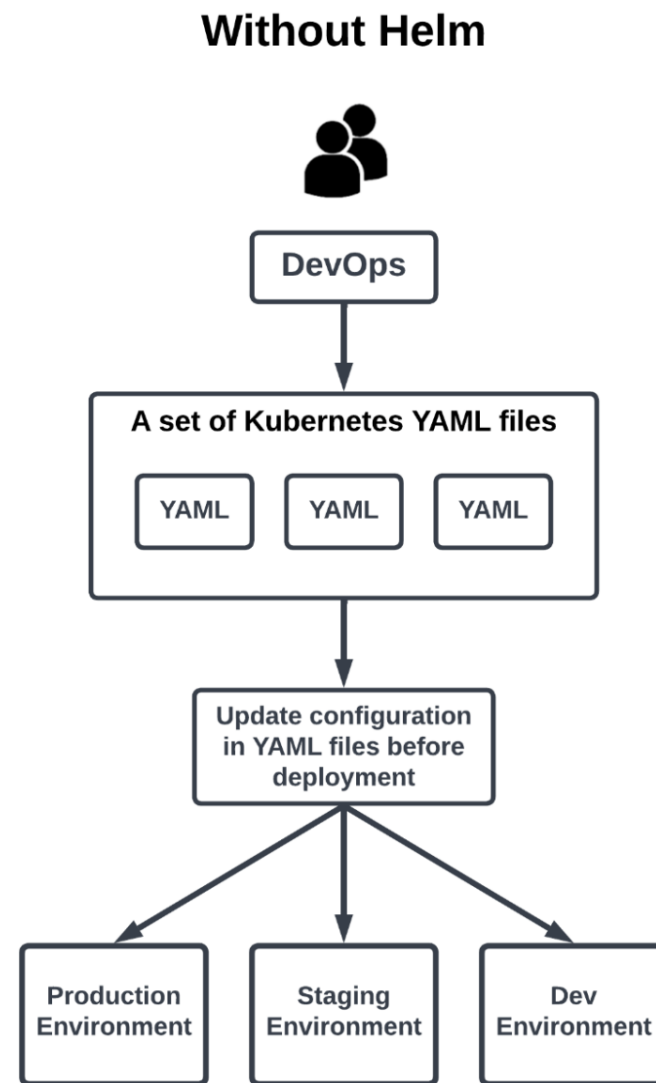


Recap:

Environment configurations separate from application



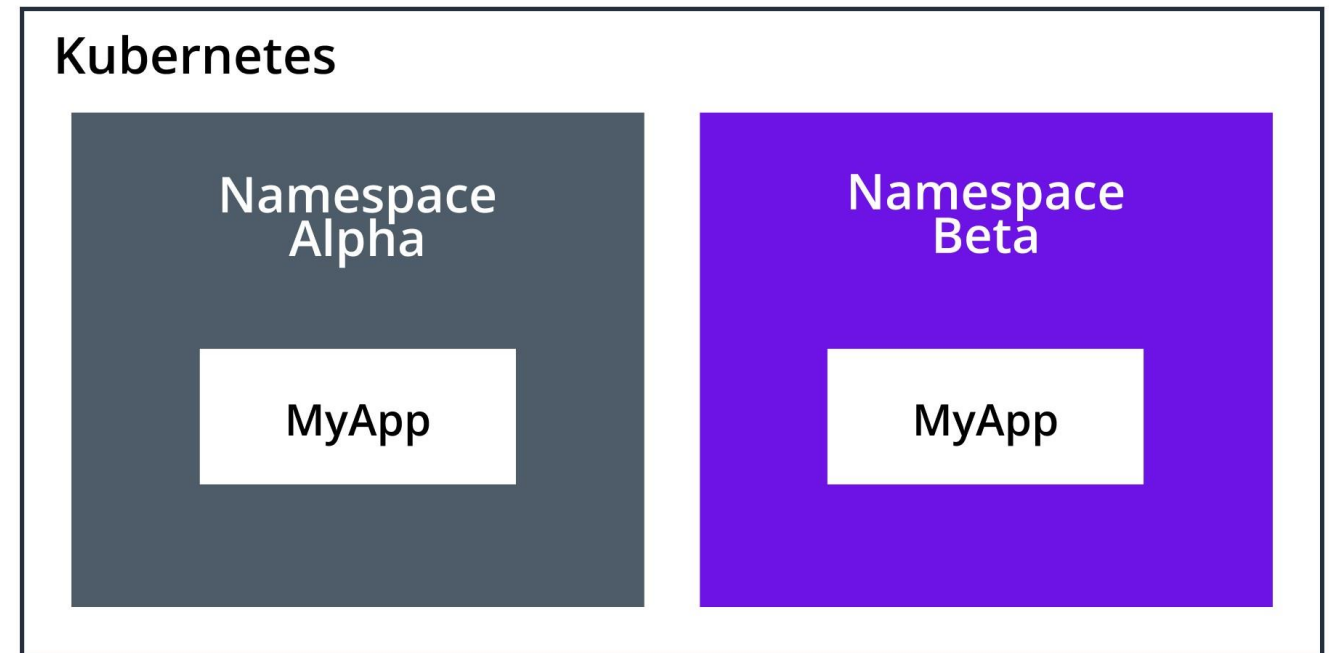
How could Helm help with Staging?



Source: <https://circleci.com/blog/what-is-helm/>

Staging and Namespaces

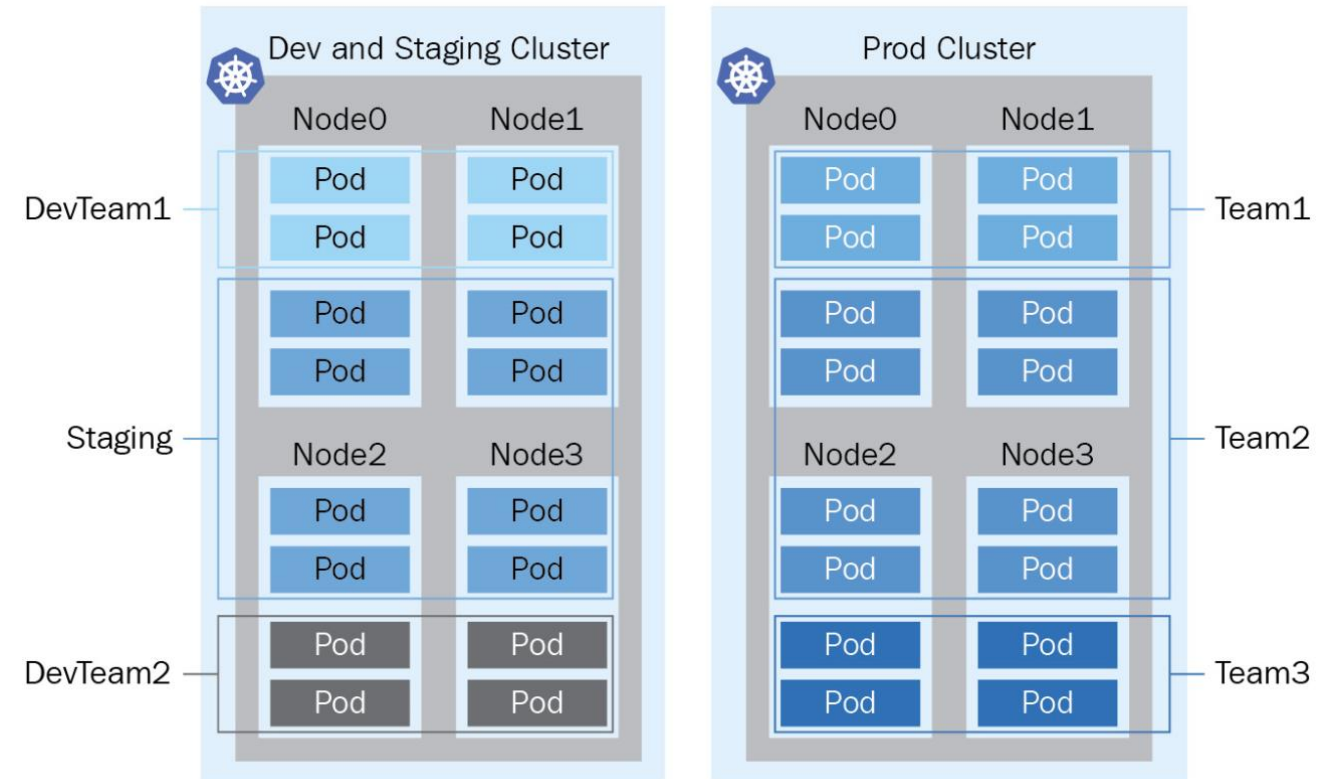
- Use the namespacing-concept for implementing different stages:
 - Stages have same network / infrastructure environment
 - Stages are accessible in a similar way
- Nevertheless:
 - Stages are isolated regarding the access, tokens, users, service accounts, resources.



Workloads are independent from infrastructure

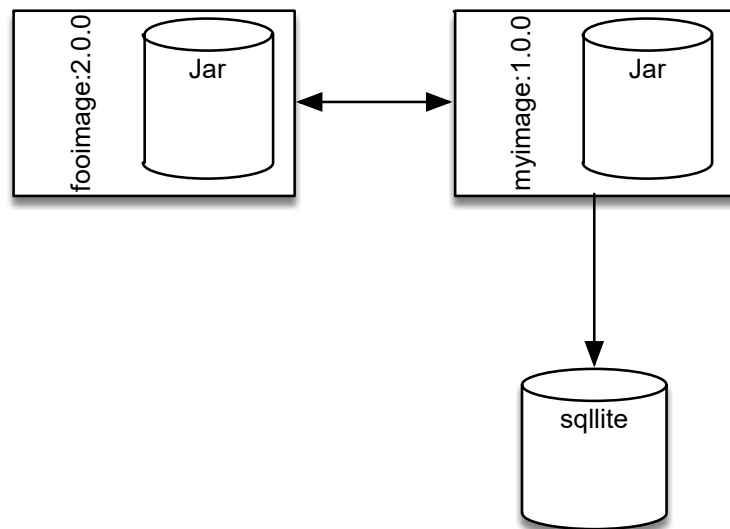
Pods might be scheduled independent of underlying infrastructure:

- Use different staging for clusters than for application and not:
dev (App) → dev (K8s)
int (App) → int (K8s)
prod (App) → prod (K8s)
- Scheduling of pods to dedicated can be directed (however not in a very fine granular way)

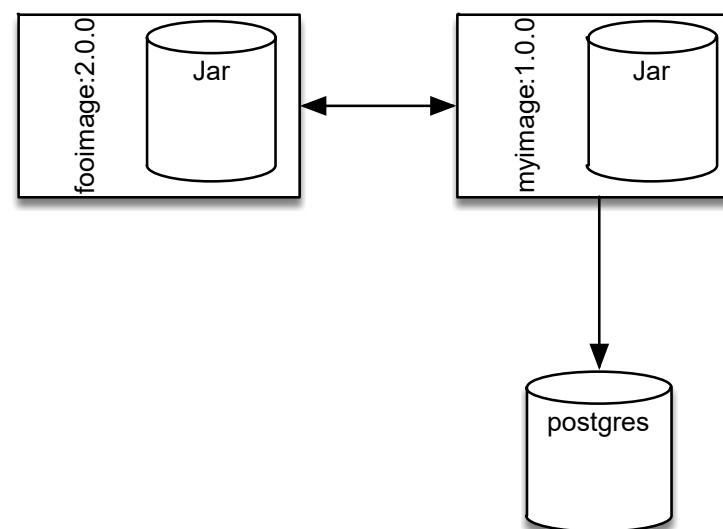


Dev/Prod Parity

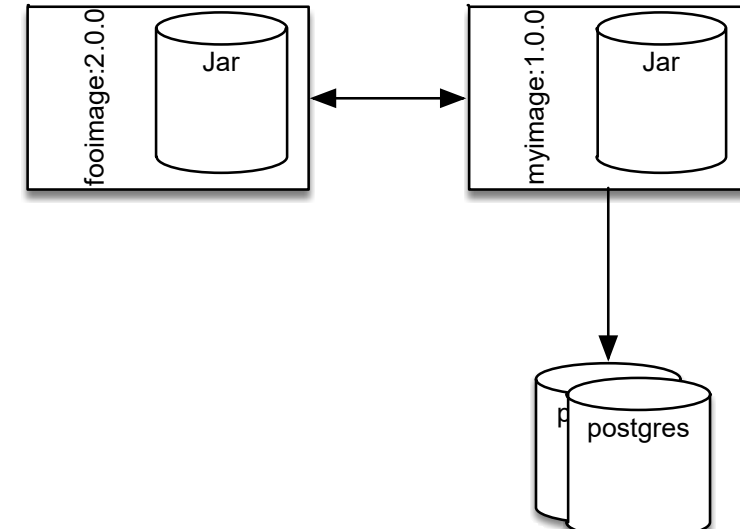
Dev



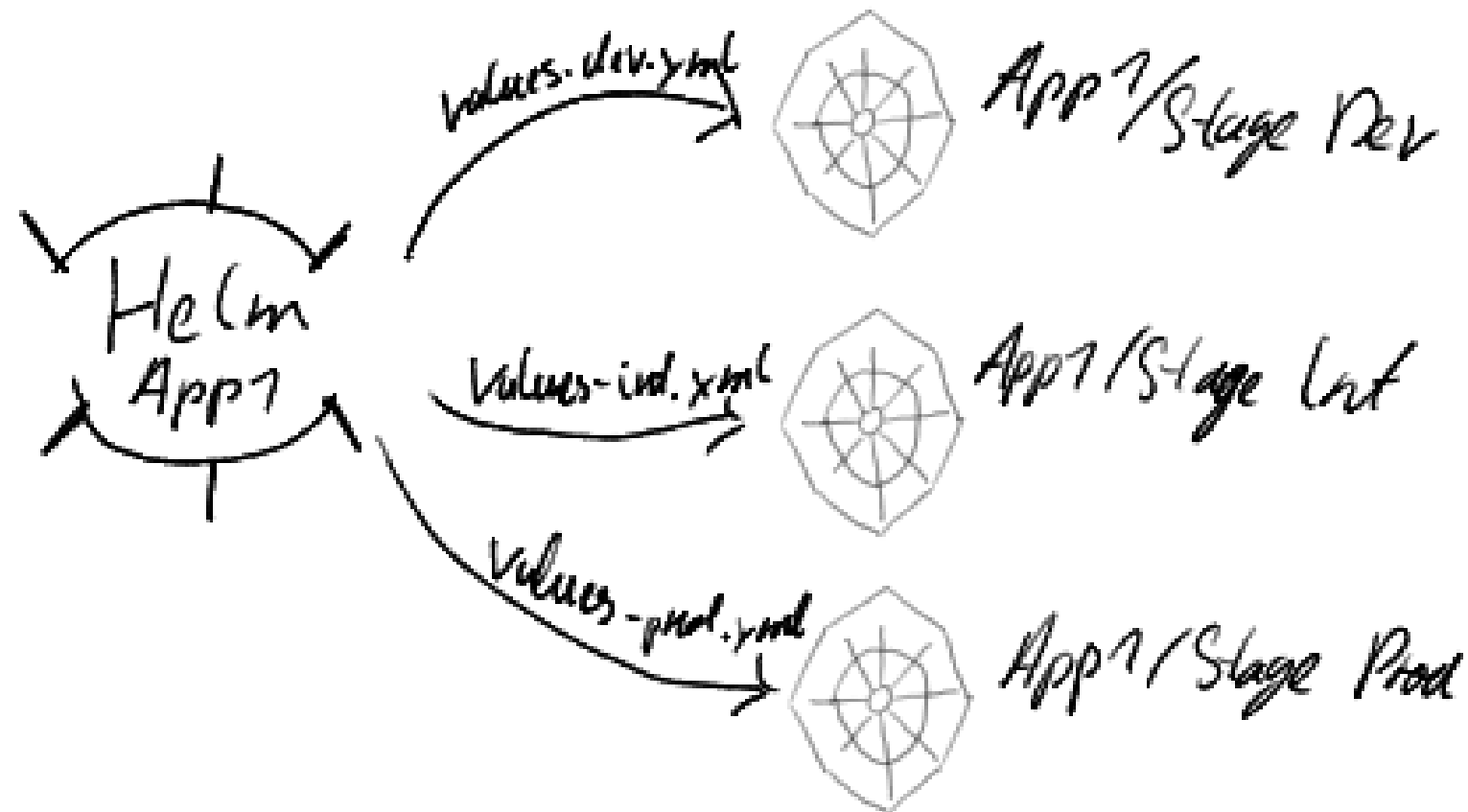
Integration



Production



Staging



Labels are important

Labels are first-class citizens in the cloud.

- All metainformation should be applied to labels/annotations.
 - Labels: identifying attributes to resources used for grouping, organizing
 - Annotations: non-identifying attributes of resources used for context to functionality or contact
- There are recommendations:
<https://kubernetes.io/docs/concepts/overview/working-with-objects/common-labels/>
- Some labels are not to be modified
 - Labels to Deployment → partially immutable
 - Labels to Pod-Template → mutable

Labels within Prometheus on K8s

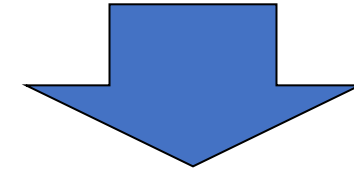
Use labels to distinguish between similar metrics

- **Metrics** should be common (latency, memory)
- **Labels** should offer filtering (endpoint, element)

Different labels are applied to a metric

- Labels from the Pod/Service itself are mapped to the metrics of the Pod/Service
- Some Labels can be derived from K8s itself:
 - Kubernetes/pod/name
 - Kubernetes/namespace
- Some Labels are set by Prometheus itself
 - Job

```
http_requests_login_total
http_requests_logout_total
http_requests_adduser_total
http_requests_comment_total
http_requests_view_total
```



```
http_requests_total{path="/login"}
http_requests_total{path="/logout"}
http_requests_total{path="/adduser"}
http_requests_total{path="/comment"}
http_requests_total{path="/view"}
```

